JavaScript In Depth

A field guide for engineers who want to *understand* the language, not just write it.

Preface

Most JavaScript guides teach you how to write code. This one teaches you how the code actually runs — what the engine does, where your mental model breaks, and why the bug you spent two hours on was inevitable given what you believed about the language.

The book is ordered deliberately: every chapter assumes the ones before it. Early chapters are things every JS developer uses daily; later chapters are things only senior engineers reach for, but depend on the earlier material. Skim the parts you know. Slow down when something surprises you — that surprise is the whole point of the book.

Each chapter follows the same shape:

- **What it is** — one or two sentences in plain language.
- **Why you care** — the real-world problem this mechanism solves.
- **How it works** — an example you can run.
- **Watch out** — the trap that catches people.

Let's begin.

Part I — Foundations

Chapter 1. Values and Types

What it is

JavaScript has exactly **eight types**. Seven are *primitives* (immutable, compared by value): `undefined`, `null`, `boolean`, `number`, `bigint`, `string`, `symbol`. The eighth is `object` (mutable, compared by reference). Functions and arrays are objects too.

Why you care

Every bug that surprises you eventually comes back to types — coercion, equality, `typeof` returning `'object'` for `null`, mutation sneaking through a reference. Knowing the type system at the bottom of your stack makes everything above it predictable.

How it works

```
typeof 42;            // 'number'
typeof 42n;           // 'bigint'
typeof 'hi';          // 'string'
typeof true;          // 'boolean'
typeof undefined;     // 'undefined'
typeof null;          // 'object'  ← historical bug, kept for compatibility
typeof Symbol();      // 'symbol'
typeof {};            // 'object'
typeof [];            // 'object'  ← arrays are objects
typeof (() => {});    // 'function' ← functions are callable objects
```

Primitives are copied when assigned. Objects are passed by *reference-value*: the variable holds a pointer.

```
let a = 10;
let b = a;
b = 99;
console.log(a); // 10 — primitives copy

let x = { n: 10 };
let y = x;
y.n = 99;
console.log(x.n); // 99 — same object
```

Watch out

- `typeof null === 'object'` is a famous footgun. Use `x === null` to check.
- `typeof undeclaredVariable` does **not** throw — it returns `'undefined'`. Every other operation on an undeclared variable is a `ReferenceError`.
- There is no "integer" type; `42` and `42.0` are the same float. Use `BigInt` when you need true integers beyond 2^{53}.

Chapter 2. Equality

What it is

JavaScript has four distinct equality algorithms. Knowing which operator uses which is the difference between code that looks right and code that *is* right.

Why you care

`Set`, `Map`, `Array.includes`, `===`, `==`, and `Object.is` do **not** agree on what "equal" means. Using the wrong one — especially when `NaN` or `-0` are in your data — leads to bugs that are impossible to spot by reading.

How it works

Algorithm	Used by	`NaN ≡ NaN`	`+0 ≡ -0`	Coerces types
Strict (`===`)	`===` , `switch`	❌	✅	❌
Abstract (`==`)	`==`	❌	✅	✅
SameValueZero	`Map` , `Set` , `Array.includes`	✅	✅	❌
SameValue	`Object.is`	✅	❌	❌

```
NaN === NaN;                        // false (Strict)
[NaN].includes(NaN);                // true  (SameValueZero)
Object.is(NaN, NaN);                // true  (SameValue)

+0 === -0;                          // true  (Strict)
Object.is(+0, -0);                  // false (SameValue)

'1' == 1;                           // true  (Abstract coerces)
'1' === 1;                          // false (Strict does not)
null == undefined;                  // true  (special case)
null === undefined;                 // false
```

Watch out

- Use `===` by default. Reach for `==` only for the `== null` idiom (matches both `null` and `undefined`).
- Can't find `NaN` in an array with `indexOf` ? Use `includes` — `indexOf` uses Strict, which thinks `NaN !== NaN` .
- `Object.is(+0, -0)` being `false` matters when bookkeeping floating-point sign bits (e.g., `1/+0 === Infinity` , `1/-0 === -Infinity`).

Chapter 3. Scope, Hoisting, and the Temporal Dead Zone

What it is

Scope is where a name is visible. JavaScript has three kinds: global, function, and block. **Hoisting** is the engine's act of reserving those names *before* any line of your code runs. The **Temporal Dead Zone (TDZ)** is the window where a `let` / `const` name exists but is not yet initialized — touching it throws.

Why you care

If you think `let` and `const` "aren't hoisted", you'll be confused when they shadow outer variables from line one. If you think `var` is block-scoped, you'll be confused when it leaks out of an `if`.

How it works

```
// var — function-scoped, hoisted, initialized to undefined
function varDemo() {
  console.log(x); // undefined (not a ReferenceError)
  var x = 1;
}

// let/const — block-scoped, hoisted but uninitialized → TDZ
function letDemo() {
  console.log(y); // ReferenceError: Cannot access 'y' before initialization
  let y = 1;
}

// Function declarations — fully hoisted (definition + name)
hello(); // works
function hello() { console.log('hi'); }

// Function expressions — only the binding is hoisted
// sayHi();              // TypeError: sayHi is not a function
var sayHi = function() {};
```

Watch out

- `var` leaks out of blocks but not functions. `for (var i = 0; i < 3; i++) setTimeout(() => console.log(i))` logs `3, 3, 3` — a single `i` shared. `let` fixes it by creating a fresh binding per iteration.
- Class declarations are in a TDZ too — you cannot use the class before its declaration line.
- Default parameter expressions live in their own scope: `function f(a = b, b = 1) {}` throws on call because `b` is in its TDZ when `a` is evaluated.

Chapter 4. Closures

What it is

A **closure** is a function that remembers the variables from the place where it was *defined*, even when it's later called somewhere else.

Why you care

Closures are how you make private state in JS, how `setTimeout` callbacks remember their context, how React hooks work internally, how module encapsulation works without classes. They also leak memory when misused.

How it works

```
function makeCounter() {
  let count = 0;                        // captured
  return {
    inc: () => ++count,
    get: () => count,
  };
}
const c = makeCounter();
c.inc(); c.inc();
c.get(); // 2

// `count` is not accessible from outside — it's private.
```

The returned functions keep a reference to `makeCounter`'s **lexical environment** on the heap. That environment survives the function return.

Use case: memoization

```
function memoize(fn) {
  const cache = new Map();            // captured — one per memoized fn
  return (arg) => {
    if (!cache.has(arg)) cache.set(arg, fn(arg));
    return cache.get(arg);
  };
}
```

Watch out

- Closures hold entire scopes, not just the variables they reference. If an inner function captures one field of a huge object, V8 often keeps the whole scope alive. In hot paths, destructure just the fields you need into local vars.
- The classic loop bug:

  ```
  for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
  for (let i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 0 1 2
  ```

Chapter 5. The `this` Binding

What it is

`this` is determined by **how a function is called**, not where it's defined — with one exception: arrow functions, which inherit `this` from their enclosing scope.

Why you care

Method references lose `this` when passed as callbacks (`elem.onclick = user.greet` — broken). Understanding the rules lets you pick the right fix (bind, arrow, or explicit wrapping) without cargo-culting.

How it works

In order of precedence:

1. `new` — `new Foo()` → `this` is the new instance.
2. **Explicit** — `fn.call(obj)`, `fn.apply(obj, args)`, `fn.bind(obj)`.
3. **Implicit** — `obj.method()` → `this` is `obj`.
4. **Default** — bare call → `undefined` in strict mode, `globalThis` in sloppy.
5. **Arrow** — lexical `this`; `call` / `apply` / `bind` cannot change it.

```
const user = {
  name: 'Alice',
  greet() { return `Hi, ${this.name}`; },
};
user.greet();                          // 'Hi, Alice'

const g = user.greet;
g();                                   // 'Hi, undefined' (sloppy) or TypeError

const bound = user.greet.bind(user);
bound();                               // 'Hi, Alice'
```

Use case: event handlers in classes

```
class Button {
  label = 'Click';
  // Arrow field → `this` is captured at instance creation
  onClick = () => console.log(this.label);
}
// new Button().onClick keeps `this` correctly even when passed elsewhere.
```

Watch out

- Arrow functions in object literals don't "inherit from the object" — there's no enclosing method. `this` in `{ fn: () => this }` is whatever surrounds the *literal*.
- `bind` returns a **new function**. Binding twice doesn't re-bind; the first wins.

Chapter 6. Coercion

What it is

Coercion is JavaScript silently converting a value to another type when an operator or API demands one. It follows a small algorithm (`ToPrimitive` → `ToNumber` / `ToString`) with surprising corners.

Why you care

Every "weird JavaScript" joke you've seen is coercion. `[] + [] === ''` , `[] + {} === '[object Object]'` , `{} + [] === 0` — all follow from one rule applied consistently. Once you know the rule, the jokes stop being confusing.

How it works

When an object needs to become a primitive, JS calls `[Symbol.toPrimitive](hint)` with `'number'` , `'string'` , or `'default'` . If that's missing, it falls back to `valueOf()` then `toString()` (or `toString()` first for hint `'string'`).

```
const money = {
  amount: 42,
  [Symbol.toPrimitive](hint) {
    if (hint === 'number') return this.amount;
    if (hint === 'string') return `$${this.amount}`;
    return `$${this.amount}`;         // default
  },
};

+money;         // 42   (hint 'number')
`${money}`;     // '$42' (hint 'string')
money + '';     // '$42' (hint 'default')
```

Use case: Date subtraction

```
const t1 = new Date();
// ...
const t2 = new Date();
const ms = t2 - t1;                    // Date's valueOf → timestamp
```

Date implements `valueOf` to return `Date.now()`, so subtraction Just Works.

Watch out

- `+` with any string coerces the other side to string: `1 + '2' === '12'`.
- `-`, `*`, `/`, `%` always coerce to numbers: `'10' - 1 === 9`.
- `==` with `{}` or `[]` triggers full coercion and often lies. Stick to `===`.

Part II — Objects

Chapter 7. Property Descriptors

What it is

Every property on an object isn't just a key/value — it has a **descriptor** controlling whether it's writable, enumerable, configurable, and whether it's a data property or an accessor (getter/setter).

Why you care

Libraries that freeze objects, browsers that lock `window` APIs, proxies that enforce invariants — all of it is descriptors. Understanding them lets you build safe APIs and explains why `Object.freeze` survives tampering.

How it works

```
const user = {};
Object.defineProperty(user, 'id', {
  value: 1,
  writable: false,                    // can't reassign
  enumerable: false,                  // hidden from for..in / Object.keys
  configurable: false,                // can't delete or reconfigure
});

user.id = 99;                         // silently ignored (sloppy) / throws (st
Object.keys(user);                    // []  — id is non-enumerable
delete user.id;                       // false; still there
```

Descriptors come in two flavors:

- **Data** — `value`, `writable`.
- **Accessor** — `get`, `set`.

```
const temp = {
  _c: 0,
  get celsius() { return this._c; },
  set celsius(v) { this._c = v; },
  get fahrenheit() { return this._c * 9/5 + 32; },
};
```

Use case: read-only API surface

```
function defineConst(obj, key, value) {
  Object.defineProperty(obj, key, { value, writable: false, configurable: false
}
```

Watch out

- Properties created by assignment (`obj.x = 1`) default to `{ writable: true, enumerable: true, configurable: true }` .
- Properties defined by class fields are `{ writable: true, enumerable: true, configurable: true }` — not hidden, not locked.
- `Object.assign` only copies **enumerable own** properties; it skips getters (it invokes them and copies the value).

Chapter 8. Getters, Setters, and Object.defineProperty

What it is

A property can be backed by functions instead of a stored value. Accessing the property runs the getter; assigning runs the setter.

Why you care

Getters give you "computed" properties that look like fields. Setters let you run validation or trigger effects on assignment — the heart of reactive frameworks like Vue 2 (before Proxy) and Svelte.

How it works

```
const person = {
  first: 'Ada',
  last: 'Lovelace',
  get full() { return `${this.first} ${this.last}`; },
  set full(v) {
    [this.first, this.last] = v.split(' ');
  },
};

person.full;                          // 'Ada Lovelace'
person.full = 'Grace Hopper';
person.first;                         // 'Grace'
```

Internally these are accessor descriptors:

```
Object.defineProperty(person, 'initials', {
  get() { return this.first[0] + this.last[0]; },
  enumerable: true,
});
```

Use case: lazy computation

```
const config = {
  get heavyThing() {
    const value = loadSlow();
    Object.defineProperty(this, 'heavyThing', { value }); // replace with data
    return value;
  },
};
```

First access computes and caches; subsequent accesses are plain property loads.

Watch out

- Getters run every time — don't do expensive work in them without memoizing.
- A setter without a getter makes reads return `undefined`, not throw. Silent bugs follow.

- Getters on prototypes see `this` as the *instance*, which is what you want — don't re-create them per instance unless they capture instance-specific closures.

Chapter 9. Prototypes and Inheritance

What it is

Every object has an internal link (`[[Prototype]]`) to another object. When you read a property the engine can't find locally, it walks the chain until it hits `null`.

Why you care

`class` is syntactic sugar over this one mechanism. Understanding the chain tells you why `instanceof` sometimes lies across iframes, why adding methods to `Array.prototype` is a terrible idea, and how `Object.create(null)` makes a safe map.

How it works

```
const animal = { eats: true };
const dog = Object.create(animal);
dog.barks = true;

dog.barks;                            // true (own)
dog.eats;                             // true (from prototype)
Object.getPrototypeOf(dog) === animal; // true

// class is the same thing with syntax
class Animal { eat() {} }
class Dog extends Animal { bark() {} }
const d = new Dog();
Object.getPrototypeOf(d) === Dog.prototype;               // true
Object.getPrototypeOf(Dog.prototype) === Animal.prototype; // true
```

Use case: safe dictionaries

`{}` inherits from `Object.prototype`, which has `hasOwnProperty`, `toString`, etc. If you store user input as keys, a key named `'toString'` collides with the inherited one.

```
const map = Object.create(null);      // no prototype — safe for arbitrary keys
map.toString = 'hello';
map.toString;                         // 'hello', not the function
```

Watch out

- Do not mutate built-in prototypes (`Array.prototype.myFn = ...`). It changes every array in the runtime and breaks libraries.
- `hasOwnProperty` on `Object.create(null)` doesn't exist. Use `Object.hasOwn(obj, key)` (ES2022) for safety.

Chapter 10. Freeze, Seal, and Preventing Extensions

What it is

Three levels of object locking: `Object.preventExtensions` (no new props), `Object.seal` (no new props + existing non-configurable), `Object.freeze` (no new, no change, no delete).

Why you care

Redux used to freeze state in dev to catch accidental mutations. Config objects you export from a library should be frozen so consumers can't mess with each other. Frozen objects also unlock minor V8 optimizations.

How it works

```
const cfg = Object.freeze({ host: 'localhost', port: 5432 });
cfg.port = 8080;                      // silently ignored (sloppy); TypeError (
cfg.user = 'admin';                   // same
delete cfg.host;                      // same
Object.isFrozen(cfg);                 // true
```

Use case: shallow immutability

```
export const LEVELS = Object.freeze(['debug', 'info', 'warn', 'error']);
```

Watch out

- `freeze` is **shallow**. Nested objects are still mutable:

  ```
  const o = Object.freeze({ inner: { n: 1 } });
  o.inner.n = 99;                     // works
  ```

  Use a recursive deep-freeze helper for true immutability.
- Freezing is one-way — there is no `unfreeze`.

Chapter 11. Symbols

What it is

A `symbol` is a primitive whose only guarantee is **uniqueness**. Two symbols with the same description are not equal.

Why you care

Symbols let libraries add metadata to user objects without name collisions. They're also how the language defines its own protocols (iteration, coercion, instance checks) — *well-known symbols*.

How it works

```
const id = Symbol('id');              // description is just a label
const id2 = Symbol('id');
id === id2;                           // false — always unique

const user = { name: 'Alice', [id]: 123 };
user[id];                             // 123
Object.keys(user);                    // ['name'] — symbols are hidden
Object.getOwnPropertySymbols(user);   // [Symbol(id)]
```

Use case: library-private state on public objects

```
const STATE = Symbol('libX.state');

export function attach(target) {
  target[STATE] = { seen: new Set() };
}
export function hasSeen(target, x) {
  return target[STATE]?.seen.has(x);
}
```

User code can't collide with `STATE` because they can't name it.

Watch out

- `Symbol()` is not a constructor — don't write `new Symbol()`.
- Symbols are hidden from `JSON.stringify`, `for..in`, and `Object.keys`. Great for privacy, confusing when debugging.
- `Symbol.for('k')` uses a **global registry** — same key returns the same symbol process-wide. Use it only when you explicitly want cross-realm shared symbols.

Chapter 12. Well-Known Symbols

What it is

Built-in symbols the language itself looks up on your objects to customize behavior. They're how you teach a class to be iterable, stringify a certain way, or be `instanceof`-compatible.

Why you care

You want your custom collection to work with `for..of` and spread? Implement `Symbol.iterator`. You want `String(myObj)` to print nicely? Use `Symbol.toStringTag`. Meta-programming starts here.

The catalogue

- `Symbol.iterator` — makes an object iterable (`for..of`, spread).
- `Symbol.asyncIterator` — `for await..of`.
- `Symbol.toPrimitive` — customize coercion (Chapter 6).
- `Symbol.toStringTag` — override the `[object X]` tag used by `Object.prototype.toString`.
- `Symbol.hasInstance` — override `instanceof`.
- `Symbol.isConcatSpreadable` — whether `Array.concat` flattens this object.
- `Symbol.species` — subclass-aware return types.
- `Symbol.matchAll`, `Symbol.replace`, etc. — hooks for string methods.

Example: custom `instanceof`

```
class Even {
  static [Symbol.hasInstance](n) {
    return typeof n === 'number' && n % 2 === 0;
  }
}
4 instanceof Even;                    // true
5 instanceof Even;                    // false
```

Example: custom `toString` tag

```
class Money {
  get [Symbol.toStringTag]() { return 'Money'; }
}
Object.prototype.toString.call(new Money()); // '[object Money]'
```

Watch out

- Overriding `Symbol.hasInstance` is a great way to confuse everyone reading your code. Use it for DSLs or matchers, not everyday classes.

Part III — Iteration and Functions

Chapter 13. Iterators and Iterables

What it is

An **iterable** is any object with a `Symbol.iterator` method that returns an **iterator**: an object with a `next()` method returning `{ value, done }`. That's the entire protocol.

Why you care

`for..of`, the spread operator (`...`), destructuring, `Array.from`, `new Map(iterable)` — all use this one protocol. Once you implement it, your type plugs into all of them for free.

How it works

```
const range = {
  from: 1,
  to: 3,
  [Symbol.iterator]() {
    let i = this.from, end = this.to;
    return {
      next: () => i <= end ? { value: i++, done: false } : { value: undefined,
    };
  },
};

for (const n of range) console.log(n); // 1 2 3
[...range];                            // [1, 2, 3]
Array.from(range, x => x * 10);        // [10, 20, 30]
```

Use case: paginate a REST API

```
async function* pages(url) {
  let next = url;
  while (next) {
    const res = await fetch(next).then(r => r.json());
    yield res.items;
    next = res.nextPage;
  }
}
for await (const batch of pages('/api/users')) process(batch);
```

Watch out

- Strings are iterable **by code point**, not code unit — so surrogate pairs (emoji) don't split: `[...'👩‍❤️‍💋‍👩']` returns the ZWJ sequence elements correctly (still more than one because of the joiner).
- Plain objects (`{}`) are **not** iterable. Use `Object.entries(obj)` or define `Symbol.iterator`.
- Iterators are usually **one-shot**. You can't `for..of` the same iterator twice — iterables are what you re-iterate.

Chapter 14. Generators

What it is

A function declared with `function*` that pauses at each `yield` and resumes when the consumer calls `next()`. The engine builds the iterator for you.

Why you care

Anything that produces a sequence lazily — infinite streams, pagination, tree traversal, state machines — reads more naturally as a generator than as a handwritten iterator.

How it works

```
function* fib() {
  let [a, b] = [0, 1];
  while (true) { yield a; [a, b] = [b, a + b]; }
}

const g = fib();
g.next().value; // 0
g.next().value; // 1
g.next().value; // 1
g.next().value; // 2
```

Generators can also **receive** values via `next(x)`:

```
function* interview() {
  const name = yield 'What is your name?';
  const age  = yield `Hi ${name}, how old are you?`;
  return `${name} is ${age}`;
}

const it = interview();
it.next();                          // { value: 'What is your name?', done: f
it.next('Alice');                   // { value: 'Hi Alice, how old are you?',
it.next(30);                        // { value: 'Alice is 30', done: true }
```

Use case: tree traversal

```
function* walk(node) {
  yield node.value;
  for (const child of node.children) yield* walk(child);
}
```

Watch out

- `return()` and `throw()` are part of the iterator protocol — `for..of` uses them to clean up on `break` or a thrown error. If you write resources (file handles), use `try/finally` inside the generator so cleanup runs.

- Generators are not optimized as aggressively as regular functions. In hot paths, a plain iterator object is sometimes faster.

Chapter 15. Async Iteration

What it is

`Symbol.asyncIterator` returns an iterator whose `next()` returns a **Promise** for `{ value, done }`. `for await..of` consumes it.

Why you care

Streaming: reading a file chunk by chunk, paging through an API, consuming a Kafka topic. You want backpressure (only ask for the next chunk when you're ready) and clean error handling — async iteration gives you both.

How it works

```
import { createReadStream } from 'node:fs';
import { createInterface } from 'node:readline';

async function* lines(path) {
  const rl = createInterface({ input: createReadStream(path) });
  for await (const line of rl) yield line;
}

for await (const line of lines('big.log')) {
  if (line.includes('ERROR')) console.log(line);
}
```

Use case: stream to DB

```
async function ingest(streamOfEvents) {
  for await (const batch of streamOfEvents) {
    await db.insertMany(batch);       // natural backpressure — next batch wait
  }
}
```

Watch out

- Use `for await..of` only inside `async` functions.
- Errors thrown inside the iterator bubble to the `for await..of` site — wrap in `try/catch`.
- Remember to `return` early if you break out mid-stream, or the source may leak a file descriptor / socket.

Part IV — Asynchronicity

Chapter 16. The Event Loop, from JavaScript's Side

What it is

JavaScript runs on **one thread** with a queue of tasks. The event loop pulls one task at a time, runs it to completion, then drains the **microtask queue**, then repeats.

Why you care

Every async bug — "why did this log before that?", "why is my UI frozen?", "why is this promise never resolving?" — comes back to the loop. Understanding the phases lets you predict order without running the code.

How it works (simplified)

```
┌──────────────┐
│  next task   │  (from the task queue: I/O callbacks, setTimeout, UI events)
└──────┬───────┘
       ↓ run to completion

┌──────────────┐
│  microtasks  │  drain ALL queued microtasks: promise reactions, queueMicrotas
└──────┬───────┘
       ↓ repeat
```

```
console.log('1');
setTimeout(() => console.log('4'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('2');

// 1, 2, 3, 4
// Script is one task. Microtasks (promise) drain before the next task (timeout
```

Use case: avoiding UI jank

Splitting a long computation into smaller chunks, yielding with `await null` or `setTimeout(fn, 0)` between them, lets the browser paint between chunks.

Watch out

- `process.nextTick` in Node is **even higher priority** than microtasks. Abusing it can starve I/O.
- A tight microtask chain (`p.then(p2).then(p3)...` recursively) can block the task queue indefinitely.

Chapter 17. Promises Internally

What it is

A Promise is a state machine with three states — **pending**, **fulfilled**, **rejected** — and a FIFO list of reactions. When it settles, every reaction is scheduled as a microtask.

Why you care

Misunderstanding Promises causes the worst async bugs: silent rejections, forgotten `await`, double `resolve` calls that do nothing, and the "promise returned from a then that returns a promise" chain that people find magical.

How it works

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve(42), 100);
});
p.then(x => x + 1)                    // returns a new promise for 43
 .then(x => Promise.resolve(x * 2))   // chain adopts — unwraps nested promise
 .then(console.log);                  // 86
```

Three rules of chaining:

1. `.then(fn)` returns a new promise for `fn`'s return value.

2. If `fn` returns a thenable, the new promise **adopts** it (unwraps one level automatically).
3. A synchronous `throw` in `fn` becomes a rejection.

Use case: parallel with `Promise.all`

```
const [user, orders, prefs] = await Promise.all([
  fetchUser(),
  fetchOrders(),
  fetchPrefs(),
]);
```

Variants: `Promise.allSettled` (no fail-fast), `Promise.race` (first to settle), `Promise.any` (first to fulfill).

Watch out

- A **forgotten** `await` returns a promise to a caller who treats it as the value. Enable lint rules (`no-floating-promises`) in TypeScript projects.
- **Double-settling** is a no-op — `resolve(1); resolve(2)` settles with 1 and ignores the second call.
- An unhandled rejection fires a `process` / `window` event; in recent Node versions it crashes the process by default.

Chapter 18. async/await

What it is

Syntactic sugar over Promises + generators. An `async` function always returns a Promise; `await` pauses the function until its Promise settles and resumes with the value (or throws the rejection).

Why you care

It makes async code read like sync code, but the mental model is different: every `await` is a microtask boundary, the stack unwinds, and errors need `try/catch` — not callbacks.

How it works

```
async function getUser(id) {
  try {
    const res = await fetch(`/u/${id}`);
    if (!res.ok) throw new Error(`${res.status}`);
    return await res.json();
  } catch (err) {
    logger.error({ err, id });
    throw err;                        // propagate
  }
}
```

Under the hood, the function is paused at each `await` — its state (local vars, pending expressions) is stored on the heap, and the runtime schedules resumption when the awaited promise settles.

Use case: sequential when needed, parallel when possible

```
// Sequential — each depends on the previous
const user = await getUser(id);
const orders = await getOrders(user.id);

// Parallel — independent
const [a, b] = await Promise.all([fa(), fb()]);
```

Watch out

- `await` inside a loop serializes — use `Promise.all(items.map(fn))` when the calls are independent.
- `return somePromise` vs `return await somePromise` differ **only** inside `try/catch`: without `await`, the rejection happens after the frame returns, so the `catch` doesn't fire.
- Top-level `await` is supported in ESM only.

Chapter 19. Microtasks vs Macrotasks

What it is

Two queues the event loop drains at different cadence. **Macrotasks** (setTimeout, I/O callbacks, UI events): one per loop turn. **Microtasks** (Promise reactions, `queueMicrotask`, `process.nextTick`): drained completely between macrotasks.

Why you care

Order-of-operations bugs. "Why did the error handler fire before the timeout I set first?" Because the handler is a microtask and the timeout is a macrotask.

How it works

```
setTimeout(() => console.log('timeout'), 0);   // macrotask
Promise.resolve().then(() => console.log('p'));// microtask
queueMicrotask(() => console.log('qm'));       // microtask
console.log('sync');

// sync, p, qm, timeout
```

Use case: defer until current work finishes without yielding to I/O

```
function scheduleFlush() {
  queueMicrotask(flush);              // runs before any timer or I/O
}
```

Watch out

- Starvation: recursive `queueMicrotask` / `process.nextTick` prevents timers and I/O from ever running.
- In browsers, the microtask queue is drained in the same loop step; in Node, `process.nextTick` runs **before** regular microtasks.

Part V — Errors

Chapter 20. Errors, Causes, and Aggregates

What it is

`Error` is a built-in class with `name`, `message`, and (non-standard but universal) `stack`. Modern specs added `cause` (wrap an original error) and `AggregateError` (multiple errors as one).

Why you care

Good error handling is half of senior engineering. Wrapping errors without losing context, distinguishing operational from programmer errors, and presenting root causes to logs are all table stakes.

How it works

```
class NotFoundError extends Error {
  constructor(msg, options) {
    super(msg, options);              // options.cause is stored automatically
    this.name = 'NotFoundError';
  }
}

try {
  fetchUser(id);
} catch (err) {
  throw new NotFoundError(`User ${id} missing`, { cause: err });
}
```

`AggregateError` is what `Promise.any` throws when *all* inputs reject:

```
try {
  await Promise.any([Promise.reject(1), Promise.reject(2)]);
} catch (e) {
  e instanceof AggregateError;        // true
  e.errors;                           // [1, 2]
}
```

Use case: typed domain errors

```
class ValidationError extends Error { constructor(issues) { super('Invalid inpu

// Middleware handler:
if (err instanceof ValidationError) return res.status(400).json({ issues: err.i
```

Watch out

- **Custom Error classes lose `instanceof` across realms** (iframes, workers). Use a sentinel property (`err.code === 'E_NOT_FOUND'`) for cross-realm checks.
- `Error.captureStackTrace(obj, constructor)` (V8 only) trims the constructor from the stack — useful in custom error hierarchies.
- Always include the original error via `cause` when wrapping — losing the root cause costs hours during incidents.

Part VI — Meta-Programming

Chapter 21. Proxy and Reflect

What it is

A `Proxy` wraps a target object and lets you intercept fundamental operations (read, write, delete, enumerate, call, construct) via **traps**. `Reflect` mirrors those operations as plain functions — use it to forward from a trap to the default behavior.

Why you care

Reactive frameworks (Vue 3), ORMs (Sequelize), mocks in tests, dynamic APIs, guard rails in development — all built on Proxy. Replacing `Object.defineProperty` approaches because Proxy is deep (handles new keys, delete, array length).

How it works

```
const target = { a: 1, b: 2 };
const logged = new Proxy(target, {
  get(t, prop, receiver) {
    console.log('get', prop);
    return Reflect.get(t, prop, receiver);
  },
  set(t, prop, value, receiver) {
    console.log('set', prop, value);
    return Reflect.set(t, prop, value, receiver);
  },
});

logged.a;                              // get a → 1
logged.a = 10;                         // set a 10
```

Traps you'll actually use: `get`, `set`, `has`, `deleteProperty`, `ownKeys`, `apply`, `construct`.

Use case: reactive state (Vue-style)

```
function reactive(obj, onChange) {
  return new Proxy(obj, {
    set(t, k, v) { t[k] = v; onChange(k, v); return true; },
  });
}

const state = reactive({ count: 0 }, (k, v) => console.log(`${k} = ${v}`));
state.count = 5;                       // logs "count = 5"
```

Watch out

- Proxies must respect **invariants**: a non-configurable, non-writable data property must return the same value from a `get` trap. Breaking an invariant throws a `TypeError`.
- Proxies are not transparent to native code: `map.set(proxy, 1)` uses the proxy as the key, not the target. Frameworks that compare identity can get confused.
- Performance: a proxy is ~10× slower than a raw access. Wrap at boundaries, not in hot loops.

Part VII — Classes

Chapter 22. Classes, Fields, and Private Members

What it is

`class` is sugar over prototypes, but modern JS adds genuinely new features on top: public fields (`x = 1`), private fields (`#x = 1`), private methods (`#fn()`), and static blocks.

Why you care

`#private` is enforced by the engine — it's actually private, unlike TypeScript's `private` which is compile-time only. Static blocks let you do complex static initialization without IIFEs or factory functions.

How it works

```
class Counter {
  #count = 0;                           // truly private
  static #instances = new Map();        // private static
  static {                              // static initialization block
    Counter.#instances.set('default', new Counter());
  }

  inc() { return ++this.#count; }
  static get(key) { return Counter.#instances.get(key); }
}

const c = new Counter();
c.inc();                               // 1
c.#count;                              // SyntaxError at parse time
```

Use case: branded checks

```
class Branded {
  #brand;
  static isBranded(x) { return #brand in x; }  // true only for real instances
}
```

Useful for distinguishing "real instance of my class" from "looks-like-a-duck".

Watch out

- Arrow methods as class fields (`onClick = () => {}`) are **per-instance** — they keep `this`, but every instance allocates a new function. Fine for event handlers on small numbers of instances; wasteful on millions.
- Class declarations are **in the TDZ** — you can't reference the class before its declaration.
- Field initializers see `this` as the instance under construction, but **not** derived fields not yet declared.

Part VIII — Modules

Chapter 23. ESM vs CommonJS

What it is

Two module systems sharing the ecosystem: the older **CommonJS** (`require` , `module.exports`) that powered Node for a decade, and the standard **ECMAScript Modules** (`import` , `export`).

Why you care

Most pain in Node tooling traces back to these two coexisting. Package authors, bundler config, `.cjs` / `.mjs` extensions, `"type": "module"` in `package.json` , default export shape — all downstream of this split.

Key differences

	CommonJS	ESM
Loader	Synchronous, on demand	Asynchronous, parsed before execution
Bindings	**Copy** of exports at `require` time	**Live** bindings (reads see current value)
`this` at top level	`module.exports`	`undefined`
Circular deps	Returns the partial `exports` at the cycle point	Works via live bindings; TDZ errors if used too early
Top-level `await`	Not supported	Supported
File extensions	Optional	Required in specifiers

How it works

```
// ESM
export const x = 1;
export default function fn() {}

import fn, { x } from './mod.js';
import * as mod from './mod.js';
```

```
// CommonJS
module.exports = { x: 1, fn: () => {} };
const { x, fn } = require('./mod');
```

Use case: the "default export shape" trap

```
// CJS module
module.exports = { a: 1 };

// ESM consumer
import mod from './x.cjs';   // mod is { a: 1 }

// ESM module
export default { a: 1 };

// CJS consumer
const mod = require('./x.mjs');  // mod is { default: { a: 1 } }  ← wrapper!
```

Watch out

- `require()` of an ESM module was a hard error until recent Node. Even today, prefer `await import()`.
- Some packages ship dual (`main` + `module` + `exports`) — misconfigure and consumers pick the wrong file. Validate with tools like `@arethetypeswrong/cli`.

Part IX — Primitives and Data

Chapter 24. Numbers and IEEE 754

What it is

All `number` values in JS are 64-bit IEEE 754 floats. There is no separate integer type. This explains `0.1 + 0.2 !== 0.3` and every other "float weirdness" question.

Why you care

If you handle money, timestamps, IDs, or scientific data, you *must* know where precision ends and how it fails.

How it works

```
Number.MAX_SAFE_INTEGER;              // 9007199254740991 (2^53 - 1)
9007199254740993 === 9007199254740992; // true — same representable float
0.1 + 0.2;                            // 0.30000000000000004
0.1 + 0.2 === 0.3;                    // false
Math.abs((0.1 + 0.2) - 0.3) < 1e-10;  // true — compare with a tolerance
```

Use case: money in cents

```
// Store amounts as integer minor units
const cents = 1099;                   // $10.99
const total = cents * quantity;       // exact
function format(c) { return `$${(c / 100).toFixed(2)}`; }
```

Never do float arithmetic on currency.

Watch out

- `JSON.parse('{"id":9999999999999999}')` produces `10000000000000000` — the parser snaps to the nearest float. Use strings for IDs > 2^{53}.
- `parseInt('08')` used to return `0` in ancient browsers because of octal interpretation. Always pass a radix: `parseInt(s, 10)`.

Chapter 25. BigInt

What it is

An integer primitive with arbitrary precision. Created with the `n` suffix (`10n`) or `BigInt(x)`.

Why you care

IDs > 2^{53}, cryptographic arithmetic, exact math — anywhere a 64-bit float lies, BigInt tells the truth.

How it works

```
const big = 9007199254740993n;
big + 1n;                              // 9007199254740994n — exact
big > 2n ** 60n;                       // false

// Mixing with Number throws
1n + 1;                                // TypeError
Number(big);                           // back to lossy float
```

Use case: Snowflake / Twitter IDs

Twitter IDs are 64-bit. Parsing as `Number` loses the bottom bits.

```
const id = BigInt(json.id_str);       // always safe
```

Watch out

- JSON doesn't serialize BigInt. Use `id.toString()` or a custom replacer/reviver.
- `Math.*` doesn't accept BigInt — use `bigint`-specific libraries for sqrt/abs etc.
- Performance: slower than Number. Don't use for counters unless you need the range.

Chapter 26. Strings and Unicode

What it is

JS strings are sequences of **UTF-16 code units**. Most characters are one unit; some (emoji, rare CJK) are two ("surrogate pairs"). Unicode code points are a separate, logical layer.

Why you care

Slicing by index, counting `.length`, iterating with a `for` loop — all operate on **code units**, and can split a character in half. Users see garbage; you don't see the bug until an emoji shows up.

How it works

```
const s = '👍';
s.length;                              // 2 — two code units (surrogate pair)
s[0];                                  // '\uD83D' — half a character
[...s].length;                         // 1 — spread uses the iterator (code poi
Array.from(s);                         // ['👍']
```

Use case: safe truncation

```
function truncate(str, maxPoints) {
  const pts = [...str];                // by code point
  return pts.length > maxPoints ? pts.slice(0, maxPoints).join('') + '…' : str;
}
```

Watch out

- Regex `.` without `/u` flag matches one code unit — also splits emoji. Always use `/u` for user-visible text.
- "Grapheme clusters" (e.g., 👨‍👩‍👦 = man + ZWJ + woman + ZWJ + boy) are not code points either. Use `Intl.Segmenter` for true user-perceived characters.

Chapter 27. Regular Expressions

What it is

A mini-language for pattern matching over strings, exposed as `RegExp` literals (`/pattern/flags`) or `new RegExp(str, flags)` . Flags change behavior: `g` (global), `i` (case-insensitive), `m` (multiline), `u` (unicode), `s` (dotall), `y` (sticky), `d` (indices).

Why you care

The wrong regex has hung more production servers than most bugs. Understanding flags and lookbehind/named groups saves you from writing 300-line parsers by hand.

How it works

```
// Named capture groups
'2026-04-21'.match(/(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})/).groups;
// { y: '2026', m: '04', d: '21' }

// Lookbehind
'price: $42'.match(/(?<=\$)\d+/);     // ['42']

// matchAll — iterate all matches with groups
for (const m of 'a1 b2'.matchAll(/(?<letter>\w)(?<num>\d)/g)) {
  console.log(m.groups);              // { letter: 'a', num: '1' }, then { lett
}

// Sticky (y) — anchor at lastIndex, never scan forward
const re = /\d+/y;
re.lastIndex = 2;
re.exec('ab123');                     // null — no match AT 2
```

Use case: tokenizer

```
function* tokens(input) {
  const patterns = [
    ['num', /\d+/y],
    ['ws',  /\s+/y],
    ['op',  /[+\-*/]/y],
  ];
  let i = 0;
  while (i < input.length) {
    for (const [name, re] of patterns) {
      re.lastIndex = i;
      const m = re.exec(input);
      if (m) { yield [name, m[0]]; i += m[0].length; break; }
    }
  }
}
```

Watch out

- **Catastrophic backtracking**: `/(a+)+b/` on `'aaaaaa…x'` takes exponential time. On untrusted input, use safe regex engines (RE2 via `re2`) or a real parser.
- A `/g` regex used with `.test()` or `.exec()` mutates `lastIndex`. Reusing the same regex across calls without resetting leads to every-other-call misses. Prefer `matchAll` or a fresh instance.

Chapter 28. JSON

What it is

`JSON` is a subset of JS — a notation for objects, arrays, strings, numbers, `true`, `false`, `null`. `JSON.stringify` serializes; `JSON.parse` deserializes.

Why you care

It's the wire format of the web. Edge cases (circular refs, `undefined`, functions, BigInt, Dates) bite in production and aren't obvious from `{a:1}` examples.

How it works

```
JSON.stringify({ a: 1, b: undefined, c: () => {}, d: Symbol() });
// '{"a":1}'  — undefined/function/symbol are DROPPED from objects
// In arrays they become null: JSON.stringify([undefined]) === '[null]'

JSON.stringify(new Date());           // '"2026-04-21T…Z"' — Date has a toJSON
JSON.stringify({ big: 1n });          // TypeError — BigInt doesn't serialize

// Circular references throw
const o = {}; o.self = o;
JSON.stringify(o);                    // TypeError
```

`toJSON` hook: any object that implements `toJSON()` has its return value used instead.

Use case: safe logging

```
function safeStringify(v) {
  const seen = new WeakSet();
  return JSON.stringify(v, (_, val) => {
    if (typeof val === 'bigint') return val.toString();
    if (typeof val === 'object' && val !== null) {
      if (seen.has(val)) return '[Circular]';
      seen.add(val);
    }
    return val;
  });
}
```

Watch out

- Numbers round to floats — big IDs lose precision. Transmit as strings.
- `JSON.parse` reviver runs bottom-up; you can reshape during parse.
- `Date` round-trips as a string, not a Date — `JSON.parse` will not revive it without a custom reviver.

Part X — Memory and Execution

Chapter 29. Memory Model and Garbage Collection

What it is

JS has automatic memory management. Objects live on the **heap**; primitives live on the stack or are inlined in objects. The GC finds objects no longer reachable from a root (globals, live stack frames, active closures) and reclaims them.

Why you care

Leaks don't mean your code uses `new`; they mean your code keeps references it no longer needs. Caches, event listeners, and long-lived closures are the usual culprits.

How it works

V8 uses a **generational, incremental, mostly concurrent** GC:

- **Young generation** (new space) — small, collected often via a copying scavenger. Most objects die young.
- **Old generation** — survivors are promoted here. Collected via mark-sweep, occasionally compacted.
- **Incremental marking** — big marks split into tiny slices between tasks, so pause times stay sub-millisecond.

Use case: finding a leak

```
// Node: run with --inspect, connect DevTools, take heap snapshots
// before and after N iterations. Diff to find retained objects.

// Or track with:
process.memoryUsage();                // { rss, heapTotal, heapUsed, external }
```

Common leak patterns

- **Unbounded caches.** Use LRU: `import LRU from 'lru-cache'`.
- **Listeners not removed.** `emitter.on('x', h)` without a matching `off`.
- **Closures capturing big objects.** `req` captured by a handler you scheduled for later, where only `req.id` was needed.
- **Module-level state.** Adding to a top-level `Map` per request without eviction.

Watch out

- Setting a variable to `null` doesn't help if another reference holds it.
- `--max-old-space-size` raises Node's heap limit; **always investigate before raising**. Usually the limit is a symptom, not the problem.

Chapter 30. Weak References

What it is

Data structures and references that **don't count** for GC: `WeakMap`, `WeakSet`, `WeakRef`, `FinalizationRegistry`.

Why you care

Caches keyed by objects shouldn't keep their keys alive. Tagging user objects with library metadata shouldn't leak. Weak refs let libraries associate data with objects without extending their lifetime.

How it works

```
const cache = new WeakMap();
function compute(obj) {
  if (!cache.has(obj)) cache.set(obj, expensive(obj));
  return cache.get(obj);
}
// When `obj` is no longer referenced anywhere else, its cache entry disappears
```

`WeakRef` : "hold this *if* it's still alive."

```
const ref = new WeakRef(largeObj);
// ...later...
const maybe = ref.deref();             // the object or undefined
```

`FinalizationRegistry` : run a callback after an object is collected.

```
const reg = new FinalizationRegistry((token) => closeFd(token));
reg.register(bufferObj, bufferObj.fd);
```

Watch out

- WeakMap is **not iterable** — you can't list keys or size. That's by design (it would expose GC timing).
- Finalization timing is **not guaranteed**. Don't rely on it for correctness, only as a safety net.
- Keys must be objects (primitives would defeat weakness) — though recent specs allow symbols.

Chapter 31. The V8 Pipeline

What it is

V8 compiles your JS through multiple tiers, starting fast and optimizing hot code.

Why you care

Knowing when V8 optimizes and when it deoptimizes explains benchmarks, guides performance work, and demystifies "why did my function suddenly get 10× slower?"

The tiers

1. **Parser** → AST.
2. **Ignition** — bytecode interpreter. Starts executing immediately.
3. **Sparkplug** — cheap baseline JIT.

4. **Maglev** — mid-tier optimizing JIT (newer).
5. **TurboFan** — top-tier optimizing JIT, aggressive inlining & speculation.

If an optimization's speculation is violated (types change, prototypes mutate), V8 **deoptimizes** — throws out the optimized code and falls back to Ignition.

Example

```
function add(a, b) { return a + b; }

// Called 10_000 times with numbers → TurboFan compiles a fast int-add
for (let i = 0; i < 10_000; i++) add(1, 2);

// Now call it with strings → speculation invalid → deopt
add('x', 'y');
```

Watch out

- Monomorphic call sites are fast. Polymorphic (2–4 shapes) are OK. **Megamorphic** (5+) are slow.
- Avoid changing object shapes after creation — shape == hidden class (next chapter).

Chapter 32. Hidden Classes and Inline Caches

What it is

Every object in V8 has an invisible "hidden class" (internally a *Map*) describing its property layout. Access by property name is compiled as a fixed-offset memory load — as long as the shape matches.

Why you care

Two objects that *look* the same but were built differently can have different hidden classes, making loops 10× slower without any visible difference in code.

How it works

```
// Same hidden class — properties added in the same order
function A() { this.x = 1; this.y = 2; }
function B() { this.x = 1; this.y = 2; }
// a and b share a shape, access to .x is a fixed offset.

// Different hidden class — properties added in different orders
function C() { this.y = 2; this.x = 1; }
// c does NOT share a shape with a — call sites that see both go polymorphic.
```

Inline caches (ICs) on each access/call site remember the shape they saw. One shape = monomorphic (fast). 2–4 = polymorphic. ≥5 = megamorphic (dictionary lookup).

Practical rules

- Initialize all properties in a constructor, in the same order.
- Don't `delete` properties on hot objects (forks the shape).
- Don't add properties to existing objects in unrelated code paths.
- Classes help: the engine sees a single constructor and the field initializer order.

Watch out

- Adding a single property conditionally (`if (admin) this.role = ...`) creates two shapes. For hot objects, initialize to `undefined` and assign later.
- Monkey-patching built-ins after warm-up throws away compiled code across the whole process.

Part XI — Binary and Shared Memory

Chapter 33. ArrayBuffer and Typed Arrays

What it is

`ArrayBuffer` is a fixed-size region of raw bytes. **Typed arrays** (`Uint8Array`, `Int32Array`, `Float64Array`, ...) are typed **views** over a buffer.

Why you care

Binary protocols, WebGL, crypto, streaming, file I/O, wasm interop — whenever you handle bytes, you reach for this.

How it works

```
const buf = new ArrayBuffer(16);      // 16 bytes of zeros
const u32 = new Uint32Array(buf);     // 4 × uint32 views
const u8  = new Uint8Array(buf);      // 16 × uint8 views over the same bytes

u32[0] = 0xDEADBEEF;
u8[0];                                // 0xEF on little-endian (your laptop)
```

`DataView` gives you explicit endianness:

```
const dv = new DataView(buf);
dv.setUint32(0, 0xDEADBEEF, /*littleEndian*/ false);
dv.getUint32(0, false);               // 0xDEADBEEF — big-endian, network order
```

Use case: parsing a binary header

```
function parseHeader(bytes) {
  const dv = new DataView(bytes.buffer, bytes.byteOffset, bytes.byteLength);
  return {
    magic: dv.getUint32(0, false),
    version: dv.getUint16(4, false),
    length: dv.getUint32(6, false),
  };
}
```

Watch out

- `buffer.slice(s, e)` copies. `new Uint8Array(buf, offset, length)` is zero-copy (a view).
- Endianness matters for protocols — always use `DataView` when reading from the wire.

Chapter 34. SharedArrayBuffer and Atomics

What it is

A buffer that can be **shared** between threads (Web Workers, Node Worker Threads) — real shared memory, not message-passed copies. `Atomics` provides the operations you need for safe concurrent access.

Why you care

Lock-free queues between workers, shared counters, high-throughput producer/consumer pipelines. The only way to do cheap thread coordination in JS.

How it works

```
// Main thread
import { Worker } from 'node:worker_threads';
const shared = new SharedArrayBuffer(4);
const counter = new Int32Array(shared);
const w = new Worker('./w.js', { workerData: shared });

// w.js
import { workerData } from 'node:worker_threads';
const counter = new Int32Array(workerData);
Atomics.add(counter, 0, 1);           // atomic increment
Atomics.notify(counter, 0, 1);        // wake one waiter
```

Coordination primitives: `Atomics.wait(arr, index, expected)` blocks until the cell changes; `Atomics.notify` wakes waiters.

Watch out

- In browsers, SharedArrayBuffer requires **cross-origin isolation** (COOP/COEP headers) due to Spectre.
- Without `Atomics`, concurrent reads/writes are undefined behavior — you will see half-updated values.

Chapter 35. Structured Clone and Transferables

What it is

`structuredClone(value)` creates a deep copy that handles Maps, Sets, Dates, RegExps, TypedArrays, cycles — things `JSON.parse(JSON.stringify(x))` silently drops or breaks.

Why you care

`postMessage` between workers, `IndexedDB` storage, reliable deep copy — all use the structured clone algorithm. Knowing what's clonable saves you from surprises.

How it works

```
const a = { d: new Date(), m: new Map([['k', 1]]), set: new Set([1]) };
const b = structuredClone(a);
b.d instanceof Date;                  // true
b.m.get('k');                         // 1

const o = {}; o.self = o;
structuredClone(o).self === structuredClone(o); // cycles handled
```

Transferables are even better: instead of copying a big buffer, you transfer ownership — source side becomes unusable, destination gets the bytes at no copy cost.

```
const big = new Uint8Array(50_000_000).buffer;
worker.postMessage(big, [big]);       // transfer the buffer
big.byteLength;                       // 0 — detached
```

Watch out

- Functions, DOM nodes, and Error objects aren't structured-clonable in all environments.
- Transfer is **destructive**. Don't touch the source after posting.

Part XII — Advanced Edges

Chapter 36. Realms and `globalThis`

What it is

A **realm** is a complete JS environment — its own globals, built-ins (Array, Object, Promise, etc.), and `globalThis`. Each iframe, each Worker, each Node `vm.createContext` is a separate realm.

Why you care

`instanceof Array` **returns false** for an array from another realm (each realm has its own `Array.prototype`). Cross-realm checks trip up libraries.

How it works

```
// Browser, in the main page
const iframe = document.createElement('iframe');
document.body.appendChild(iframe);
const Arr2 = iframe.contentWindow.Array;
const a = new Arr2();
a instanceof Array;                  // false!
Array.isArray(a);                    // true — realm-aware check
```

Safe cross-realm checks:

- `Array.isArray(x)`
- `Object.prototype.toString.call(x) === '[object Date]'`
- Sentinel property on your own classes: `err.code === 'E_FOO'`

Watch out

- `globalThis` refers to the current realm's global. Don't cache "the" globalThis in a library — it breaks when the library is loaded from different realms.

Chapter 37. `eval` — Direct vs Indirect

What it is

`eval(str)` runs the string as JS. It's deeply discouraged for user input (injection, performance), but even when safe, there are two modes with different scope rules.

Why you care

Interview bait. Also: tooling (bundlers, REPLs, dev tools) uses it, and knowing the rules makes some bugs less mysterious.

How it works

```
// Direct eval: uses caller's scope
function f() {
  const local = 1;
  return eval('local');               // 1
}

// Indirect eval: runs at global scope
function g() {
  const local = 1;
  return (0, eval)('local');          // ReferenceError: local is not defined
}
```

The aliasing trick `(0, eval)(...)` forces indirect eval — the one you actually want if you must eval at all.

Watch out

- Strict mode disables many of eval's tricks (can't create new vars in the enclosing scope).
- Modern code uses `new Function(...)` (always indirect, isolated scope) or `vm.runInNewContext` (Node) instead.

Chapter 38. Intl — Internationalization

What it is

The `Intl` namespace provides locale-aware formatting: numbers, currencies, dates, collation, pluralization, list formatting, relative times, and grapheme segmentation.

Why you care

Correct formatting for users worldwide, without shipping locale data or maintaining regex hacks. The browser/Node already has ICU data; use it.

Examples

```
new Intl.NumberFormat('de-DE', { style: 'currency', currency: 'EUR' }).format(1
// '1.234,56 €'

new Intl.DateTimeFormat('en-US', { dateStyle: 'full', timeStyle: 'short' }).for
// 'Tuesday, April 21, 2026 at 10:15 AM'

new Intl.RelativeTimeFormat('en').format(-3, 'day');  // '3 days ago'

// Grapheme segmentation — true user-perceived characters
const seg = new Intl.Segmenter('en', { granularity: 'grapheme' });
[...seg.segment('👩‍❤️a')].length;   // 2 — the family emoji counts as one graphem
```

Watch out

- Constructing an `Intl.*` instance is expensive — cache them if used in a loop.
- Locale data comes from ICU; small Node/browser builds (e.g., `node-lite`) may have reduced locale coverage.

Chapter 39. Decorators (Stage 3/4)

What it is

Functions that modify classes, methods, fields, or accessors at definition time. Decorators landed as stage 3 and are supported by TypeScript (new-style in 5.x) and modern bundlers.

Why you care

Replaces a lot of boilerplate in frameworks (NestJS, MobX, TypeORM). Less pattern-matching, more declarative code.

How it works

```
function log(_target: any, context: ClassMethodDecoratorContext) {
  const name = String(context.name);
  return function (this: any, ...args: any[]) {
    console.log(`→ ${name}`, args);
    // @ts-ignore
    const r = _target.apply(this, args);
    console.log(`← ${name}`, r);
    return r;
  };
}

class Math2 {
  @log
  double(n: number) { return n * 2; }
}
```

Watch out

- The "legacy" decorators (TypeScript's old `experimentalDecorators`) and the stage-3 spec have different APIs. Pick one per project.
- Decorator evaluation order is top-to-bottom for declaration but bottom-to-top for application — they wrap like onion layers.

Appendix A — Performance Cheat Sheet

1. **Monomorphism beats cleverness.** Keep hot call sites seeing one shape.
2. **Initialize properties in a fixed order.** Don't `delete`.
3. **Avoid `arguments` in hot code.** Use rest params.
4. **Cache Intl formatters.** Construction is expensive.
5. **Use Maps for dynamic keys**, objects for fixed structure.
6. **Avoid `try/catch` in hot inner loops** (pre-TurboFan versions couldn't optimize through them; TurboFan can, but it still pessimizes in practice — measure).
7. **`for` with `let i` is still faster than `.forEach`** in most cases by a small margin. Use whichever you read faster; measure when it matters.
8. **Buffers and typed arrays** beat string concatenation for binary work.
9. **Profile with `--prof` (Node) / the DevTools performance tab.** Measure, don't guess.

Appendix B — Gotchas Quick Reference

Symptom	Likely cause
`this` is undefined in a callback	Lost method binding; use arrow or `bind`
`0.1 + 0.2 !== 0.3`	IEEE 754 rounding — compare with tolerance
`NaN !== NaN`	Strict equality spec; use `Number.isNaN` or `includes`
Array of `undefined` after `new Array(n).map(...)`	`new Array(n)` creates *sparse* — use `Array.from({length:n}, fn)`
`for..in` iterates over inherited string keys	Use `for..of`, `Object.entries`, or `Object.keys`
`JSON.stringify` dropped a field	It was `undefined`, a function, or a symbol
A big number changes after JSON round-trip	Lost precision beyond 2^{53} — use strings or BigInt
Regex matches skip every other call	`/g` + `.exec` / `.test` leaves `lastIndex` set
`instanceof` returns false for an object that should match	Cross-realm; use `Array.isArray` / sentinel property
Memory grows, never shrinks	Unbounded cache, listener leak, or closure holding too much

---

Afterword

JavaScript punishes guesses and rewards patience. When something surprises you, resist the urge to add a workaround until you can explain what happened. The explanation will almost always point to a chapter in this book — usually an earlier one than you expected.

Good luck. Read the spec when you're stuck. Remember that the language is smaller than it looks: a handful of mechanisms, composed relentlessly.